Inkolo Connect full domain deployment guide

Prepared for publishing Inkolo Connect on a real domain

This guide explains how to publish Inkolo Connect on a real domain from start to finish.

It is written for a non-developer. You do not need to understand the code to follow it, but you do need access to:

- your domain account, where you register the domain;
- GitHub, where the project files are stored;
- Railway, where the backend and database run;
- Netlify, where the website frontend runs.

1. The simple picture

Inkolo Connect has three main parts:

1. Website frontend

- This is the part people see.
- It includes the login screen, dashboards, buttons, service pages, and forms.
- It will be hosted on Netlify.
- Example final address: `https://YOURDOMAIN.co.za`

2. Backend API

- This is the engine behind the website.
- It handles login, users, subscriptions, profiles, uploads, and saved information.
- It will be hosted on Railway.
- Example final address: `https://api.YOURDOMAIN.co.za`

3. Database

- This stores the real data.
- It will be a MySQL database on Railway.
- Users do not visit the database directly. Only the backend talks to it.

The final setup should look like this:

```
User opens website
        |
        v
https://YOURDOMAIN.co.za              Netlify frontend
        |
        v
https://api.YOURDOMAIN.co.za          Railway backend API
        |
        v
Railway MySQL database                Saved user/platform data
```

2. Your important names

Before you start, choose the domain structure.

Recommended:

```
Website: https://YOURDOMAIN.co.za
Website with www: https://www.YOURDOMAIN.co.za
Backend/API: https://api.YOURDOMAIN.co.za
```

Example:

```
Website: https://inkoloconnect.co.za
Website with www: https://www.inkoloconnect.co.za
Backend/API: https://api.inkoloconnect.co.za
```

In this guide, replace `YOURDOMAIN.co.za` with your real domain.

3. Project files you must keep

The project folder is:

```
C:\Users\johan\Documents\Codex\2026-06-25\ca\inkolo-connect
```

These are the important files and folders:

```
frontend
backend
backend/database
scripts
Dockerfile
netlify.toml
package.json
.dockerignore
.gitignore
.npmrc
```

What each one does:

- `frontend`: the website that users see.
- `backend`: the server/API that powers the app.
- `backend/database`: the database setup files.
- `scripts`: helper scripts used during deployment.
- `Dockerfile`: tells Railway how to build and start the backend.
- `netlify.toml`: tells Netlify how to build the frontend.
- `package.json`: tells hosting services how to install and run the project.

Do not upload these manually:

- `node_modules`
- temporary files
- old test ZIP files

For a real production website, do not depend on:

```
backend/data/platform-store.json
```

That file is for demo-style storage. A proper domain deployment should use MySQL.

4. Accounts you need

You need these accounts:

1. GitHub

- Stores the Inkolo Connect code.
- Current repository:
`https://github.com/Johannduranki/Inkolo-connect`

2. Railway

- Runs the backend.
- Runs the MySQL database.

3. Netlify

- Runs the frontend website.

4. Domain registrar

- This is where you buy/register your domain.
- Examples: GoDaddy, Domains.co.za, Namecheap, Cloudflare, etc.

5. Step 1: Make sure the latest code is on GitHub

Open GitHub and go to:

```
https://github.com/Johannduranki/Inkolo-connect
```

Check that the repository contains the project files:

- `frontend`
- `backend`
- `Dockerfile`
- `netlify.toml`
- `package.json`

If these are visible in GitHub, continue.

If the files are missing, the code still needs to be uploaded before you deploy.

6. Step 2: Create the Railway project

1. Open Railway.
2. Create a new project.
3. Choose deploy from GitHub.
4. Select:

```
Johannduranki/Inkolo-connect
```

5. Railway should find the `Dockerfile`.
6. Let Railway create the backend service.

At this point, the backend may fail until the database and variables are added. That is normal.

7. Step 3: Add the MySQL database on Railway

Inside the same Railway project:

1. Click New.
2. Choose Database.
3. Choose MySQL.
4. Wait for the MySQL service to finish creating.
5. Open the MySQL service.
6. Go to Variables.

You need these database values:

```
MYSQLHOST
MYSQLPORT
MYSQLUSER
MYSQLPASSWORD
MYSQLDATABASE
```

Railway may show them with slightly different names, but they mean:

- host: where the database lives;
- port: usually `3306`;
- user: the database username;
- password: the database password;
- database: the database name.

Keep this browser tab open because you will copy these values into the backend service.

8. Step 4: Add backend variables on Railway

Open the Railway backend/API service.

Go to Variables.

Add these variables one by one:

```
PORT=3000
FRONTEND_ORIGIN=https://YOURDOMAIN.co.za
FRONTEND_ORIGINS=https://YOURDOMAIN.co.za,https://www.YOURDOMAIN.co.za
DB_HOST=paste Railway MySQL host here
DB_PORT=paste Railway MySQL port here
DB_USER=paste Railway MySQL username here
DB_PASSWORD=paste Railway MySQL password here
DB_NAME=paste Railway MySQL database name here
JWT_EXPIRES_IN=1h
ALLOW_DEMO_AUTH=false
FORCE_DEMO_MODE=false
```

You also need two private secret values:

```
ID_PEPPER=put a long private random value here
JWT_SECRET=put a different long private random value here
```

Important:

- Do not use short words like `password123`.
- Do not use the same value for `ID_PEPPER` and `JWT_SECRET`.
- Do not share these values publicly.

Simple example of the kind of value you want:

```
Inkolo-2026-private-secret-value-change-me-92837465
```

Use your own private values, not the example above.

9. Step 5: Deploy the Railway backend

After the variables are saved:

1. Railway should ask you to apply/redeploy changes.
2. Click Apply Changes or Redeploy.
3. Wait for the deployment to finish.

The backend starts by running the database setup automatically.

The database files are:

```
backend/database/001_create_users.sql
backend/database/002_add_membership.sql
backend/database/003_create_service_subscriptions.sql
backend/database/004_create_service_applications.sql
backend/database/005_create_platform_tables.sql
backend/database/006_create_legal_acceptances.sql
backend/database/007_add_church_branding.sql
```

When Railway says the deployment is successful, the backend is running.

10. Step 6: Test the Railway backend before adding the domain

Railway gives the backend a temporary public address.

It may look similar to:

```
https://inkolo-connect-api-production.up.railway.app
```

Open this in your browser:

```
https://your-railway-backend-address/api/health
```

Expected result:

- You should see a small health/check response.
- This means the backend is awake.

If this does not work:

1. Go back to Railway.
2. Open the backend service.
3. Check Deployments.
4. Open the latest failed deployment.
5. Look for missing variables or database connection errors.

Most backend failures are caused by:

- wrong database host;
- wrong database password;
- missing `JWT_SECRET`;
- missing `ID_PEPPER`;
- database service not running yet.

11. Step 7: Add the backend custom domain

Now connect:

```
api.YOURDOMAIN.co.za
```

to the Railway backend.

In Railway:

1. Open the backend service.
2. Go to Settings or Networking.
3. Find Domains.
4. Add a custom domain:

```
api.YOURDOMAIN.co.za
```

5. Railway will show DNS instructions.
6. Open your domain registrar account.
7. Add the DNS record Railway gives you.

The DNS record may be a `CNAME`.

It will look roughly like this:

```
Type: CNAME
Name: api
Value: something Railway gives you
```

Use the exact value Railway gives you.

After adding the DNS record:

1. Return to Railway.
2. Wait for the custom domain to verify.
3. Railway should create HTTPS/SSL automatically.

When ready, test:

```
https://api.YOURDOMAIN.co.za/api/health
```

Do not continue until this works.

12. Step 8: Create the Netlify frontend site

Open Netlify.

1. Click Add new site.
2. Choose Import from Git.
3. Choose GitHub.
4. Select:

```
Johannduranki/Inkolo-connect
```

5. Netlify should read `netlify.toml`.

Confirm these settings:

```
Build command:
node scripts/netlify-build.mjs

Publish directory:
frontend/dist/duranki-login/browser
```

13. Step 9: Add the frontend backend URL on Netlify

Before deploying the Netlify site, add this environment variable:

```
BACKEND_URL=https://api.YOURDOMAIN.co.za
```

This tells the website where the backend lives.

Without this, the frontend may open but login and service data may fail.

Then deploy the Netlify site.

14. Step 10: Test the Netlify temporary website

Netlify gives the site a temporary address first.

It may look similar to:

```
https://some-name.netlify.app
```

Open:

```
https://some-name.netlify.app/login
```

Test:

1. The login page opens.
2. The page does not show blank white screen.
3. Login reaches a dashboard.
4. Dashboard refresh still works.

If login fails, check:

- Railway backend is online.
- `https://api.YOURDOMAIN.co.za/api/health` works.
- Netlify has `BACKEND_URL=https://api.YOURDOMAIN.co.za`.
- Railway has `FRONTEND_ORIGINS` including the Netlify temporary URL if you are testing before the final domain.

For temporary testing, Railway may need:

```
FRONTEND_ORIGINS=https://YOURDOMAIN.co.za,https://www.YOURDOMAIN.co.za,https://you
r-netlify-temp-name.netlify.app
```

After changing this, redeploy Railway.

15. Step 11: Add the main domain to Netlify

In Netlify:

1. Open the Inkolo Connect site.
2. Go to Domain management.

3. Add custom domain:

```
YOURDOMAIN.co.za
```

4. Also add:

```
www.YOURDOMAIN.co.za
```

Netlify will show DNS records.

Go to your domain registrar and add the DNS records exactly as Netlify shows them.

Common records may look like:

```
Type: A
Name: @
Value: Netlify IP address
```

and:

```
Type: CNAME
Name: www
Value: your Netlify site address
```

Use Netlify's exact instructions, because they may vary.

16. Step 12: Wait for SSL/HTTPS

Both domains need HTTPS:

```
https://YOURDOMAIN.co.za
https://api.YOURDOMAIN.co.za
```

Netlify and Railway usually create SSL certificates automatically.

You may need to wait. It can take a few minutes, sometimes longer.

Do not start final testing until both addresses use `https://` without a browser warning.

17. Step 13: Final Railway frontend origin settings

After your real domain works, go back to Railway.

Open the backend service variables.

Set:

```
FRONTEND_ORIGIN=https://YOURDOMAIN.co.za
FRONTEND_ORIGINS=https://YOURDOMAIN.co.za,https://www.YOURDOMAIN.co.za
```

If you also want to keep the Netlify temporary address working, include it:

```
FRONTEND_ORIGINS=https://YOURDOMAIN.co.za,https://www.YOURDOMAIN.co.za,https://you
r-netlify-temp-name.netlify.app
```

Then redeploy the Railway backend.

18. Step 14: Final production test

Open:

```
https://YOURDOMAIN.co.za/login
```

Test these areas:

1. Login page loads.
2. Member login works.
3. Admin dashboard opens.
4. Bishop dashboard opens.
5. Pastor dashboard opens.
6. Member dashboard opens.
7. Service-provider dashboard opens.
8. Service subscription button works.
9. Member profile changes save.
10. Community pages open.
11. Messages or contact information save.
12. Uploads work if the feature is being used.
13. Legal terms pages open.
14. Logout works.
15. Login works again after logout.
16. Refreshing the dashboard does not break the page.
17. `https://api.YOURDOMAIN.co.za/api/health` works.

19. Step 15: Turn off demo mode for real use

For a real live app, Railway should have:

```
ALLOW_DEMO_AUTH=false
FORCE_DEMO_MODE=false
```

This means the app uses the real MySQL database.

If these are set to `true`, the app may behave like a demo and not like a proper production system.

20. What is already functional after this

After these steps, the app should have:

- real public website domain;
- real backend API domain;
- HTTPS security;
- Railway backend service;
- Railway MySQL database;
- login flow;
- dashboards;
- service subscription screens;
- saved app data in MySQL;
- frontend/backend connection.

21. What may still need provider accounts

Some services can appear in the app but may not perform real-world transactions until separate provider accounts are connected.

These may include:

- payments;
- airtime and data purchases;
- prepaid electricity;
- SMS;
- WhatsApp;
- email;
- banking;
- wallet services;
- fibre service activation;
- third-party funeral or insurance providers.

This is normal. The website can be live before all commercial provider integrations are connected.

22. Troubleshooting

The website opens but login does not work

Check:

- Railway backend is running.
- `https://api.YOURDOMAIN.co.za/api/health` works.
- Netlify has `BACKEND_URL=https://api.YOURDOMAIN.co.za`.
- Railway `FRONTEND_ORIGINS` includes your frontend domain.
- Railway was redeployed after variable changes.

The backend deployment fails

Check Railway backend variables:

- `DB_HOST`
- `DB_PORT`
- `DB_USER`
- `DB_PASSWORD`
- `DB_NAME`
- `ID_PEPPER`
- `JWT_SECRET`

Also check that the MySQL service is running.

The domain does not work yet

Check:

- DNS records were added exactly as Netlify/Railway requested.
- You waited for DNS to update.
- HTTPS certificate is active.
- You did not accidentally add the API domain to Netlify or the website domain to Railway.

Correct split:

```
YOURDOMAIN.co.za      -> Netlify
www.YOURDOMAIN.co.za  -> Netlify
api.YOURDOMAIN.co.za  -> Railway backend
```

The app works on Netlify address but not on your domain

Check Railway:

```
FRONTEND_ORIGIN=https://YOURDOMAIN.co.za
FRONTEND_ORIGINS=https://YOURDOMAIN.co.za,https://www.YOURDOMAIN.co.za
```

Redeploy Railway after changing those.

The app works, but saved data disappears

This usually means the backend is in demo mode.

Check Railway:

```
ALLOW_DEMO_AUTH=false
FORCE_DEMO_MODE=false
```

Also confirm the backend is connected to MySQL.

23. Final go-live checklist

Use this list before showing the app to other people:

- GitHub repository has the latest Inkolo Connect files.
- Railway project exists.
- Railway backend service is online.
- Railway MySQL service is online.
- Railway backend has all required variables.
- Railway backend custom domain works.
- `https://api.YOURDOMAIN.co.za/api/health` works.
- Netlify site exists.
- Netlify has `BACKEND_URL=https://api.YOURDOMAIN.co.za`.
- Netlify frontend custom domain works.
- `https://YOURDOMAIN.co.za/login` works.
- HTTPS is active for website and API.
- Demo mode is off.
- Login works.
- Dashboards work.
- Service subscription flow works.
- Saved data remains after logout/login.

24. Recommended order if you want help doing it live

If you want to do this carefully, follow this order:

1. Register the domain.
2. Confirm the latest files are on GitHub.
3. Create Railway MySQL.
4. Create Railway backend.
5. Add Railway variables.
6. Test Railway API health.
7. Add `api.YOURDOMAIN.co.za` to Railway.
8. Create Netlify site from GitHub.
9. Add `BACKEND_URL` in Netlify.
10. Test temporary Netlify website.
11. Add `YOURDOMAIN.co.za` to Netlify.
12. Add `www.YOURDOMAIN.co.za` to Netlify.
13. Update Railway frontend origins.
14. Final test.